# Breast Cancer Analysis Using K-K-Nearest Neighbor Classification

In [1]:
```
import pandas as pd
import numpy as np
```

In [2]:
```
file = "C:\\Users\\DajahV01\\Desktop\\DHF Assignment\\K-Nearest Neighbor Classifi

df = pd.read_csv(file)
```

In [3]:
```
df.head()
```

Out[3]:

| | id | diagnosis | radius_mean | texture_mean | perimeter_mean | area_mean | smoothness |
|---|---|---|---|---|---|---|---|
| **0** | 842302 | M | 17.99 | 10.38 | 122.80 | 1001.0 | 0 |
| **1** | 842517 | M | 20.57 | 17.77 | 132.90 | 1326.0 | 0 |
| **2** | 84300903 | M | 19.69 | 21.25 | 130.00 | 1203.0 | 0 |
| **3** | 84348301 | M | 11.42 | 20.38 | 77.58 | 386.1 | 0 |
| **4** | 84358402 | M | 20.29 | 14.34 | 135.10 | 1297.0 | 0 |

5 rows × 32 columns

In [4]:
```
df.shape
```

Out[4]:
```
(569, 32)
```

In [5]:
```
1 df.isnull().sum()
```

```
texture_se                0
perimeter_se              0
area_se                   0
smoothness_se             0
compactness_se            0
concavity_se              0
concave_points_se         0
symmetry_se               0
fractal_dimension_se      0
radius_worst              0
texture_worst             0
perimeter_worst           0
area_worst                0
smoothness_worst          0
compactness_worst         0
concavity_worst           0
concave_points_worst      0
symmetry_worst            0
fractal_dimension_worst   0
dtype: int64
```

In [6]:
```
1 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   id                       569 non-null    int64
 1   diagnosis                569 non-null    object
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave_points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave_points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave_points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB
```

In [7]:
```
df.drop(['id'], axis = 1, inplace = True)
```

In [8]:
```
df.diagnosis = [1 if each == "M" else 0 for each in df.diagnosis]
```

In [9]:
```
y = df.loc[:,"diagnosis"]
y
```

Out[9]:
```
0      1
1      1
2      1
3      1
4      1
      ..
564    1
565    1
566    1
567    1
568    0
Name: diagnosis, Length: 569, dtype: int64
```

In [10]:
```
X = df.loc[:, df.columns != "diagnosis"]
X
```

Out[10]:

| | radius_mean | texture_mean | perimeter_mean | area_mean | smoothness_mean | compactnes |
|---|---|---|---|---|---|---|
| **0** | 17.99 | 10.38 | 122.80 | 1001.0 | 0.11840 | |
| **1** | 20.57 | 17.77 | 132.90 | 1326.0 | 0.08474 | |
| **2** | 19.69 | 21.25 | 130.00 | 1203.0 | 0.10960 | |
| **3** | 11.42 | 20.38 | 77.58 | 386.1 | 0.14250 | |
| **4** | 20.29 | 14.34 | 135.10 | 1297.0 | 0.10030 | |
| **...** | ... | ... | ... | ... | ... | |
| **564** | 21.56 | 22.39 | 142.00 | 1479.0 | 0.11100 | |
| **565** | 20.13 | 28.25 | 131.20 | 1261.0 | 0.09780 | |
| **566** | 16.60 | 28.08 | 108.30 | 858.1 | 0.08455 | |
| **567** | 20.60 | 29.33 | 140.10 | 1265.0 | 0.11780 | |
| **568** | 7.76 | 24.54 | 47.92 | 181.0 | 0.05263 | |

569 rows × 30 columns

In [11]:
```
type(X)
```

Out[11]:
```
pandas.core.frame.DataFrame
```

In [12]:
```
type(y)
```

Out[12]:
```
pandas.core.series.Series
```

In [14]:
```
print(X.shape, y.shape)
```

```
(569, 30) (569,)
```

In [15]:
```
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
%matplotlib inline
```

In [16]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, rando
```

In [17]:
```
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(455, 30)
(114, 30)
(455,)
(114,)
```

In [23]:
```
y_train.value_counts()
```

Out[23]:
```
diagnosis
0    278
1    177
Name: count, dtype: int64
```

In [24]:
```
print(y_train.value_counts()/X_train.shape[0])
```

```
diagnosis
0    0.610989
1    0.389011
Name: count, dtype: float64
```

In [28]:
```
df['diagnosis'].value_counts() / len(df) * 100
```

Out[28]:
```
diagnosis
0    62.741652
1    37.258348
Name: count, dtype: float64
```

In [29]:
```
print(y_test.value_counts()/X_test.shape[0])
```

```
diagnosis
0    0.692982
1    0.307018
Name: count, dtype: float64
```

In [31]:
```
import warnings
warnings.filterwarnings('always')
warnings.filterwarnings('ignore')

from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris

```

In [32]:
```
knn = KNeighborsClassifier(n_neighbors = 5)
```

In [33]:
```
knn.fit(X_train, y_train)
```

Out[33]:
```
KNeighborsClassifier
KNeighborsClassifier()
```

In [35]:

```
train_predictions = knn.predict(X_train)
test_predictions = knn.predict(X_test)

### Train data accuracy
from sklearn.metrics import accuracy_score, f1_score
from sklearn.metrics import confusion_matrix

print("TRAIN Conf Matrix : \n", confusion_matrix(y_train, train_predictions))
print("\nTRAIN DATA ACCURACY",accuracy_score(y_train,train_predictions))
print("\nTrain data f1-score for class '1'",f1_score(y_train,train_predictions,po
print("\nTrain data f1-score for class '2'",f1_score(y_train,train_predictions,po

### Test data accuracy
print("\n\n---------------------------------------\n\n")

print("TEST Conf Matrix : \n", confusion_matrix(y_test, test_predictions))
print("\nTEST DATA ACCURACY",accuracy_score(y_test,test_predictions))
print("\nTest data f1-score for class '1'",f1_score(y_test,test_predictions,pos_
print("\nTest data f1-score for class '2'",f1_score(y_test,test_predictions,pos_
```

```
TRAIN Conf Matrix :
 [[270   8]
 [ 17 160]]

TRAIN DATA ACCURACY 0.945054945054945

Train data f1-score for class '1' 0.9275362318840581

Train data f1-score for class '2' 0.9557522123893805


--------------------------------------


TEST Conf Matrix :
 [[76  3]
 [ 3 32]]

TEST DATA ACCURACY 0.9473684210526315

Test data f1-score for class '1' 0.9142857142857143

Test data f1-score for class '2' 0.9620253164556962
```

In [ ]:
```
1
```

In [ ]:
```
1
```